Joining tables

Monday, June 8, 2026

Today: Monday June 8

Before class

- **Perusall Assignment**: *DC* §11.1–11.3

Plan for today

- LEGO joins activity
- Why data gets split across tables
- Keys
- `left_join()`
- `join_by()`
- `anti_join()` and `semi_join()`

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* Ch. 19 · *DC* Ch. 11 · dplyr reference

The four joins

What a join does

A **join** combines two tables by matching rows on a shared column, which is called the **key**.

	Role
Left table	Your primary data
Right table	The lookup table (columns you want to attach)

The join type controls which rows survive when a row in one table has no match in the other.

In R: `left_join(x, y, join_by(...))`.

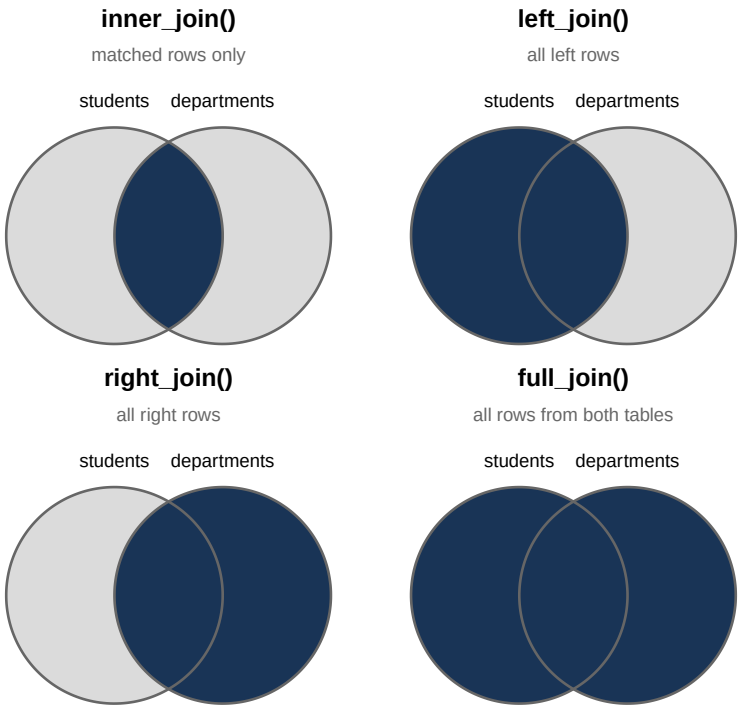
- `x` is the left table
- `y` is the right table.

Brickerly College Joins

The data

Each person gets 6 student bricks + 4 department bricks + 1 baseplate.

inner · left · right · full



Activity overview

Each of you should have two sets of bricks, a baseplate, and a handout showing the two corresponding datasets. The two kinds of bricks are:

- **Department bricks** (STAT, ECON, PSY, ENG): the **right table** in R
- **Student bricks** (A–F, each with a declared major): the **left table** in R

We'll build **4 joins**, one at a time. Each slide describes a scenario and asks you to physically build the answer on your baseplate. After a minute or two, I'll show the answer on the projector.

> **Note**
>
> **Stacking convention:** department brick at the base, student bricks above it. This is just a physical arrangement for the activity; it has nothing to do with which table is "left" or "right" in R.

Join 1

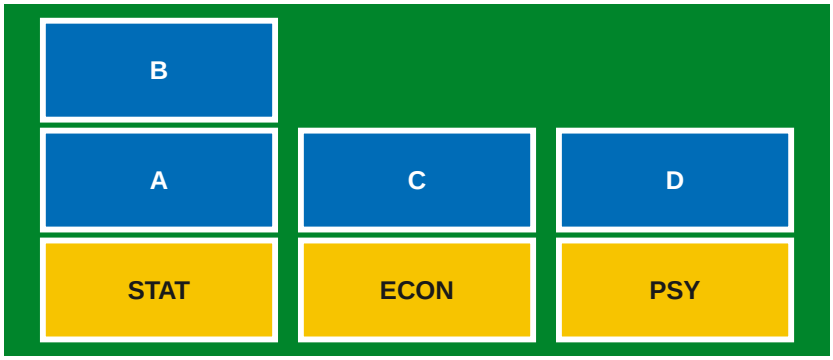
"Brickerly College is printing a glossy brochure of Student-Department success stories. They only want to feature complete pairs. If a student in their directory no longer belongs to one of their college's four departments, or if one of the four departments currently has no students, they will leave them off the brochure."

Left table: `students` · **Right table:** `departments`

Build this join on your baseplate.

INNER JOIN: keep only rows that match in *both* tables.

Inner join



On the plate: A with STAT, B with STAT, C with ECON, D with PSY.

Off the plate: E (undeclared), F (major not in Brickerly College), ENG (no students). The inner join only keeps mutual matches.

```
inner_join(students, departments, join_by(major == code))
```

Join 2

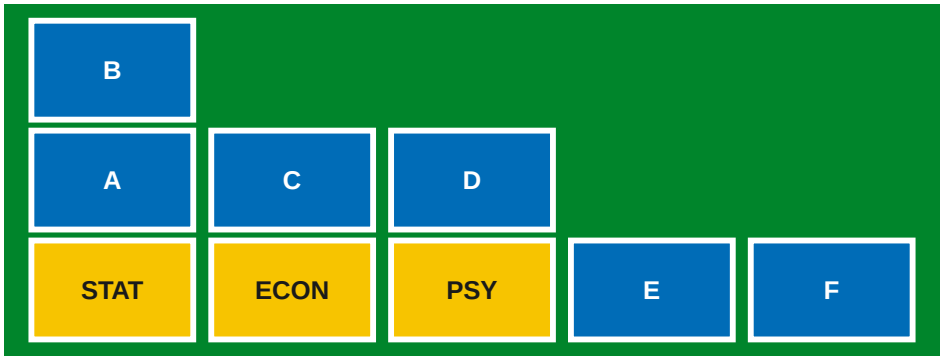
"Academic advising needs to send a check-in email to every single student on the roster. We want to include their department if we have it, but even if they are undeclared or switched to a major outside of our college, they must be on the email."

Left table: `students` · **Right table:** `departments`

Build this join on your baseplate.

LEFT JOIN: keep every row from the left table; attach matches from the right where they exist.

Left join



On the plate: A with STAT, B with STAT, C with ECON, D with PSY, E solo (no declared major) and F solo (major not in Brickerly College)

Off the plate: ENG (no students in the major)

```
left_join(students, departments, join_by(major == code))
```

Join 3

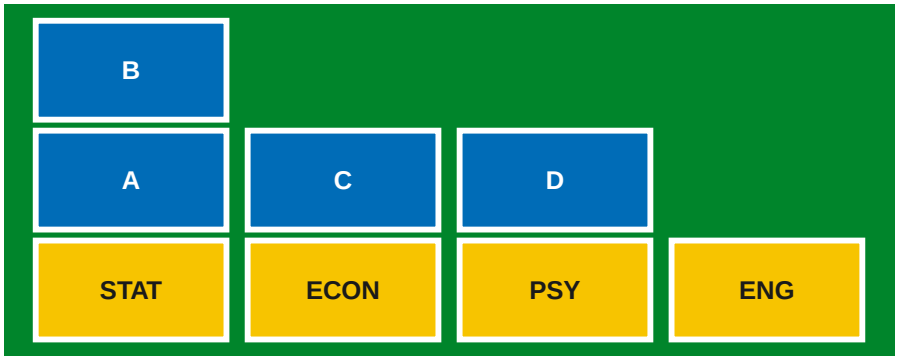
"Brickerly College is setting up booths for the Academic Fair. All four departments in the college must get a booth, even if they currently have zero students enrolled. If they have students, place them at the booth."

Left table: `students` · **Right table:** `departments`

Build this join on your baseplate.

RIGHT JOIN: keep every row from the right table; attach matches from the left where they exist.

Right join



On the plate: A with STAT, B with STAT, C with ECON, D with PSY, ENG with no students

Off the plate: E (no major declared) and F (major not in Brickerly College)

```
right_join(students, departments, join_by(major == code))
```

Join 4

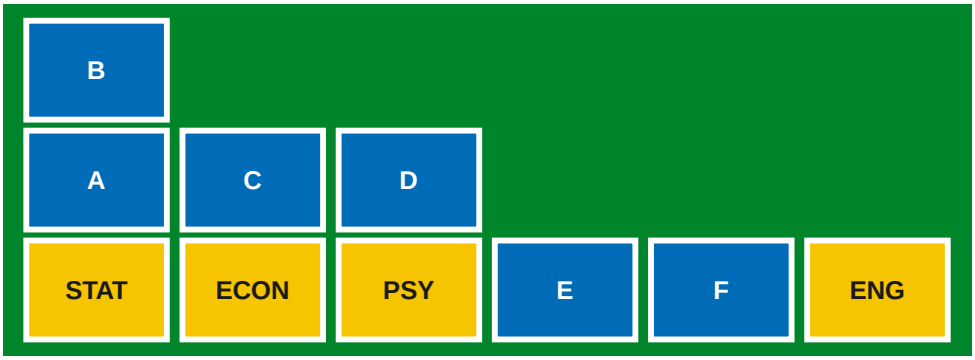
"The Provost has demanded a total audit of Brickerly College. Every student and every department must appear on the board at least once, regardless of whether they have a match or not."

Left table: `students` · **Right table:** `departments`

Build this join on your baseplate.

FULL JOIN: keep every row from both tables; fill in `NA` where there's no match.

Full join



On the plate: A with STAT, B with STAT, C with ECON, D with PSY, E solo (no major declared), F solo (major not in Brickerly College), ENG solo (no students in the major)

Off the plate: no bricks

```
full_join(students, departments, join_by(major == code))
```

Recap

Join	On the baseplate	Left off
Inner	Matched stacks	E, F, ENG
Left	Stacks + E solo + F solo	ENG
Right	Stacks + ENG solo	E, F
Full	Stacks + E solo + F solo + ENG solo	nothing

In dplyr, the corresponding four joins are `inner_join()`, `left_join()`, `right_join()`, and `full_join()`. `left_join()`, where you keep everything in your main table and attach what you can from the right, will likely be the one you use the most.

Why multiple tables?

Data is often split up on purpose

The `flights` dataset from nycflights13 has 336,776 rows, one per flight. Each row has a `carrier` column (`"UA"`, `"AA"`, etc.), but not the airline's full name.

The full names can be found in a separate `airlines` dataset:

```
airlines
```

```
# A tibble: 16 x 2
   carrier name
   <chr>   <chr>
 1 9E      Endeavor Air Inc.
 2 AA      American Airlines Inc.
 3 AS      Alaska Airlines Inc.
 4 B6      JetBlue Airways
 5 DL      Delta Air Lines Inc.
 6 EV      ExpressJet Airlines Inc.
 7 F9      Frontier Airlines Inc.
 8 FL      AirTran Airways Corporation
 9 HA      Hawaiian Airlines Inc.
10 MQ      Envoy Air
11 OO      SkyWest Airlines Inc.
12 UA      United Air Lines Inc.
13 US      US Airways Inc.
14 VX      Virgin America
15 WN      Southwest Airlines Co.
16 YV      Mesa Airlines Inc.
```

Storing the name separately is on purpose here. The name belongs to the airline, not to each individual flight. Keeping it in one place means you only update it once, and you don't repeat "United Air Lines Inc." hundreds of thousands of times unnecessarily.

Joins bring the tables back together

A **join** matches rows from one table to rows in another and combines them side by side:

```
flights |>
  select(month, day, carrier, flight, origin, dest) |>
  left_join(x = _, y = airlines, join_by(carrier))
```

```
# A tibble: 336,776 x 7
   month   day carrier flight origin dest  name
   <int> <int> <chr>    <int> <chr>  <chr> <chr>
 1     1     1 UA        1545 EWR    IAH   United Air Lines Inc.
 2     1     1 UA        1714 LGA    IAH   United Air Lines Inc.
 3     1     1 AA        1141 JFK    MIA   American Airlines Inc.
 4     1     1 B6         725 JFK    BQN   JetBlue Airways
 5     1     1 DL         461 LGA    ATL   Delta Air Lines Inc.
 6     1     1 UA        1696 EWR    ORD   United Air Lines Inc.
 7     1     1 B6         507 EWR    FLL   JetBlue Airways
 8     1     1 EV        5708 LGA    IAD   ExpressJet Airlines Inc.
 9     1     1 B6          79 JFK    MCO   JetBlue Airways
10     1     1 AA         301 LGA    ORD   American Airlines Inc.
# i 336,766 more rows
```

The `name` column is now attached to each flight. The airline name appears once in `airlines`, but as many times as needed in the result.

Keys

The columns that connect tables

A **key** is a column (or set of columns) that identifies rows and links tables together.

A **primary key** uniquely identifies each row in its own table:

- `airlines$carrier`: each two-letter code appears exactly once
- `airports$faa`: each three-digit airport code appears exactly once
- `planes$tailnum`: each tail number appears exactly once

When more than one key is needed, the key is called a **compound key**.

A **foreign key** refers to a primary key in another table:

- `flights$carrier` refers to `airlines$carrier`
- `flights$dest` (and `flights$origin`) refer to `airports$faa`
- `flights$tailnum` refers to `planes$tailnum`

The key columns often share the same name in both tables, which makes the join straightforward. But when they don't, you need to tell R the distinct names of the two key columns; otherwise, it will not know how to merge the two tables.

How the nycflights13 tables connect

Here, `flights` is the central table, and the other tables each add detail about a piece of a flight:

Column(s) in `flights`	Connects to	Table
`carrier`	`carrier`	`airlines`
`tailnum`	`tailnum`	`planes`
`origin`, `dest`	`faa`	`airports`
`origin` + `time_hour`	`origin` + `time_hour`	`weather`

Both `origin` and `dest` refer to the same column in `airports`. You can join on either one depending on what you're asking.

Checking a primary key

Before joining, it's worth verifying that a column is actually a primary key. For a column to be a primary key, there must be no duplicates. To count and look for any `n > 1`, you can run the following code:

```
airlines |>
  count(carrier) |>
  filter(n > 1)
```

```
# A tibble: 0 x 2
# i 2 variables: carrier <chr>, n <int>
```

The result here is an empty tibble, which confirms `carrier` is unique in `airlines`. If you had gotten rows back, that would've meant that the column wasn't a primary key and the join could produce more rows than you expected.

left_join()

The most common join

`left_join(x, y)` keeps every row from `x` and attaches matching columns from `y`. If there's no match in `y`, the corresponding entries in the new columns are assigned an `NA` value.

```r
flights |>
  select(month, day, tailnum, carrier) |>
  left_join(x = _, y = planes |> select(tailnum, manufacturer, model),
            join_by(tailnum))
```

```
# A tibble: 336,776 x 6
   month   day tailnum carrier manufacturer     model
   <int> <int> <chr>   <chr>   <chr>            <chr>
 1     1     1 N14228  UA      BOEING           737-824
 2     1     1 N24211  UA      BOEING           737-824
 3     1     1 N619AA  AA      BOEING           757-223
 4     1     1 N804JB  B6      AIRBUS           A320-232
 5     1     1 N668DN  DL      BOEING           757-232
 6     1     1 N39463  UA      BOEING           737-924ER
 7     1     1 N516JB  B6      AIRBUS INDUSTRIE A320-232
 8     1     1 N829AS  EV      CANADAIR         CL-600-2B19
 9     1     1 N593JB  B6      AIRBUS           A320-232
10     1     1 N3ALAA  AA      <NA>             <NA>
# i 336,766 more rows
```

> **Tip**
>
> Typically, we use `left_join()` when we want to add information to a main table. The left table will stay completely intact since you're just pulling in extra columns from the right.

When there's no match: NA

Some tail numbers in `flights` don't appear in `planes`. `left_join()` keeps those rows anyway and fills the new columns with `NA`. The `N3ALAA` tailnumber is one example:

```r
flights |>
  filter(tailnum == "N3ALAA") |>
  select(month, day, tailnum, carrier) |>
  left_join(x = _, y = planes |> select(tailnum, manufacturer, model),
            join_by(tailnum))
```

```
# A tibble: 63 x 6
   month   day tailnum carrier manufacturer model
```

```
   <int> <int> <chr>   <chr>   <chr>        <chr>
 1     1     1 N3ALAA  AA      <NA>         <NA>
 2     1     2 N3ALAA  AA      <NA>         <NA>
 3     1     3 N3ALAA  AA      <NA>         <NA>
 4     1     7 N3ALAA  AA      <NA>         <NA>
 5     1     8 N3ALAA  AA      <NA>         <NA>
 6     1    16 N3ALAA  AA      <NA>         <NA>
 7     1    20 N3ALAA  AA      <NA>         <NA>
 8     1    22 N3ALAA  AA      <NA>         <NA>
 9    10    11 N3ALAA  AA      <NA>         <NA>
10    10    14 N3ALAA  AA      <NA>         <NA>
# i 53 more rows
```

What's going on here is that the flight happened, but there are no plane records. This albeit incomplete information is still valuable information to have; we want the missing information to show up as `NA` rather than being silently left out of the joined table.

join_by()

Matching column names

When the key has the same name in both tables, `join_by()` is simple:

```
flights |>
  left_join(x = _, y = airlines, join_by(carrier)) # carrier == carrier

flights |>
  left_join(x = _, y = planes, join_by(tailnum)) # tailnum == tailnum
```

dplyr prints which columns it matched on, so you can make sure it's working correctly.

Different column names

`airports$faa` is the primary key, but `flights` uses `origin` and `dest` to refer to airports. You tell R which columns correspond with `==` inside `join_by()`:

```
flights |>
  select(month, day, origin, dest) |>
  left_join(x = _, y = airports |> select(faa, name, lat, lon),
            join_by(dest == faa))
```

```
# A tibble: 336,776 x 7
   month   day origin dest  name                              lat   lon
   <int> <int> <chr>  <chr> <chr>                           <dbl> <dbl>
 1     1     1 EWR    IAH   George Bush Intercontinental     30.0 -95.3
 2     1     1 LGA    IAH   George Bush Intercontinental     30.0 -95.3
 3     1     1 JFK    MIA   Miami Intl                       25.8 -80.3
 4     1     1 JFK    BQN   <NA>                             NA    NA
 5     1     1 LGA    ATL   Hartsfield Jackson Atlanta Intl  33.6 -84.4
 6     1     1 EWR    ORD   Chicago Ohare Intl               42.0 -87.9
 7     1     1 EWR    FLL   Fort Lauderdale Hollywood Intl   26.1 -80.2
 8     1     1 LGA    IAD   Washington Dulles Intl           38.9 -77.5
 9     1     1 JFK    MCO   Orlando Intl                     28.4 -81.3
10     1     1 LGA    ORD   Chicago Ohare Intl               42.0 -87.9
# i 336,766 more rows
```

`join_by(dest == faa)` tells R to match rows where `dest` in the left table equals `faa` in the right table.

Common pitfall

If you omit `join_by()`, dplyr matches on **all** shared column names. This can go wrong, and the dangerous part is that it will not necessarily throw an error.

`flights` and `planes` both have a `year` column, but `flights$year` is the flight year and `planes$year` is the year the plane was built:

```
# joins on both tailnum and year
# almost certainly not what you want
flights |>
  select(year, month, day, tailnum) |>
  left_join(x = _, y = planes)
```

```
Joining with `by = join_by(year, tailnum)`
```

```
# A tibble: 336,776 x 11
    year month   day tailnum type  manufacturer model engines seats speed engine
   <int> <int> <int> <chr>   <chr> <chr>        <chr>   <int> <int> <int> <chr>
 1  2013     1     1 N14228  <NA>  <NA>         <NA>       NA    NA    NA <NA>
 2  2013     1     1 N24211  <NA>  <NA>         <NA>       NA    NA    NA <NA>
 3  2013     1     1 N619AA  <NA>  <NA>         <NA>       NA    NA    NA <NA>
 4  2013     1     1 N804JB  <NA>  <NA>         <NA>       NA    NA    NA <NA>
```

```
 5  2013     1     1 N668DN  <NA>  <NA>          <NA>       NA    NA    NA <NA>
 6  2013     1     1 N39463  <NA>  <NA>          <NA>       NA    NA    NA <NA>
 7  2013     1     1 N516JB  <NA>  <NA>          <NA>       NA    NA    NA <NA>
 8  2013     1     1 N829AS  <NA>  <NA>          <NA>       NA    NA    NA <NA>
 9  2013     1     1 N593JB  <NA>  <NA>          <NA>       NA    NA    NA <NA>
10  2013     1     1 N3ALAA  <NA>  <NA>          <NA>       NA    NA    NA <NA>
# i 336,766 more rows
```

Because almost no plane was built in 2013 with a matching tailnum, the result is full of `NA`s. Always specify `join_by()` when columns share a name but mean different things.

Other mutating joins

left, inner, right, full

All four mutating joins add columns from the right table. The difference is which rows survive. Here are the same four cases you just built with bricks:

Join	Rows kept
`left_join()`	all rows from the **left** table
`inner_join()`	only rows with a match in **both** tables
`right_join()`	all rows from the **right** table
`full_join()`	all rows from **either** table

`left_join()` is the right choice for adding additional variables to a main table. `inner_join()` is useful when you only want rows that can be fully matched on both sides.

left vs. inner join

```
# left_join: keeps all 336,776 flights; unmatched rows get NA
nrow(flights |>
       left_join(x = _, y = planes, join_by(tailnum))
     )
```

```
[1] 336776
```

```r
# inner_join: drops any flight with no matching plane record
nrow(flights |>
       inner_join(x = _, y = planes, join_by(tailnum))
     )
```

```
[1] 284170
```

About 50,000 flights disappear with the inner join. Whether that's the appropriate move depends on your question.

Filtering joins

semi_join() and anti_join()

Filtering joins don't add columns. They filter rows based on whether a match exists in another table:

Join	Keeps…
`semi_join(x, y)`	rows in `x` that **have** a match in `y`
`anti_join(x, y)`	rows in `x` that **don't** have a match in `y`

Here, only the columns of `x` appear in the result, and nothing from `y` is added. This is helpful for checking what's present or absent in one table relative to another.

anti_join()

Which flights have a tail number that doesn't appear in `planes` at all?

```r
flights |>
  anti_join(x = _, y = planes, join_by(tailnum)) |>
  distinct(tailnum)
```

```
# A tibble: 722 x 1
   tailnum
   <chr>
 1 N3ALAA
 2 N3DUAA
 3 N542MQ
```

```
 4 N730MQ
 5 N9EAMQ
 6 N532UA
 7 N3EMAA
 8 N518MQ
 9 N3BAAA
10 N3CYAA
# i 712 more rows
```

Over 700 unique tail numbers in `flights` have no matching record in `planes`. This information is probably good to know before you join in plane characteristics for an analysis.

semi_join()

`semi_join()` filters by membership, keeping only the rows from `x` that appear in `y`. For example, this function can be used to answer the question: which airports actually appear as destinations in `flights`?

```
airports |>
  semi_join(x = _, y = flights, join_by(faa == dest))
```

```
# A tibble: 101 x 8
   faa   name                              lat    lon   alt    tz dst   tzone
   <chr> <chr>                           <dbl>  <dbl> <dbl> <dbl> <chr> <chr>
 1 ABQ   Albuquerque International Sunport  35.0 -107.   5355    -7 A     Ameri~
 2 ACK   Nantucket Mem                     41.3  -70.1    48    -5 A     Ameri~
 3 ALB   Albany Intl                       42.7  -73.8   285    -5 A     Ameri~
 4 ANC   Ted Stevens Anchorage Intl        61.2 -150.    152    -9 A     Ameri~
 5 ATL   Hartsfield Jackson Atlanta Intl   33.6  -84.4  1026    -5 A     Ameri~
 6 AUS   Austin Bergstrom Intl             30.2  -97.7   542    -6 A     Ameri~
 7 AVL   Asheville Regional Airport        35.4  -82.5  2165    -5 A     Ameri~
 8 BDL   Bradley Intl                      41.9  -72.7   173    -5 A     Ameri~
 9 BGR   Bangor Intl                       44.8  -68.8   192    -5 A     Ameri~
10 BHM   Birmingham Intl                   33.6  -86.8   644    -6 A     Ameri~
# i 91 more rows
```

Only 101 of the 1,458 airports in `airports` are actual destinations in this dataset.

What we covered today

- **Why joins**: data is split by design; joins combine it on demand without duplicating storage
- **Keys**: a primary key uniquely identifies rows in its table; a foreign key points to a primary key elsewhere
- `left_join()`: keeps all left rows, pulls in columns from the right; unmatched rows get `NA`
- `join_by()`: always specify the key; use `a == b` when column names differ between tables
- **Natural join pitfall**: shared column names get joined automatically and can silently give wrong results
- **Mutating join comparison**: left keeps all left rows; inner keeps only matches; right and full less common
- `anti_join()`: rows with no match; useful for finding data quality gaps
- `semi_join()`: rows with a match; useful for filtering by membership in another table

Wrap-up & looking ahead

Tomorrow (Tuesday): importing data from CSV files, Excel, and web URLs, plus a look at factors vs. strings.

Upcoming Deadlines

- **Tue 6/9, before class**: *DC* §16.1–16.2, 16.4–16.5
- **Mon 6/15, 11:59 p.m.**: HW 4
- **Thu 6/11, in class**: Exam

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 19 · *DC* Ch. 11 · dplyr reference